

COMP 6231: Distributed Systems

Assignment 1

Architectural Design and Trade-Off Analysis of a Real-Time Ride-Sharing Platform Backend

Author: Travis Holt
Student ID: 40351596
Instructor: Dr. Reza Mirsalari
Date: July 1, 2026

1 Introduction

1.1 Problem Statement

"...architect the backend infrastructure of a real-time ride-sharing platform similar to Uber or Lyft, using principles from distributed systems. Your task is to analyze and design a distributed backend system that supports high availability, low latency, and data consistency under high concurrency."

You must integrate and analyze the following core architectural components

- Database Sharding (Hash-based, Range-based, or Consistent Hashing)
- Replication and Consistency Models (Eventual, Strong, Quorum-based)
- Load Balancing (Layer 4 vs. Layer 7)

1.2 System Requirements

1.2.1 Functional Requirements

- Drivers broadcast live location and availability.
- Riders send trip requests that are matched with nearby drivers.
- The system supports live matching, route calculation, and payment processing.
- Data includes user accounts, trip histories, real-time GPS data, and pricing rules.

1.2.2 Non-Functional Requirements

- Global scale support (multi-region architecture assumed).
- Sub-200ms latency for ride-matching response in 90% of requests.
- High read/write throughput for hotspots (e.g., downtown during peak hours).

1.3 Report Organization

The proposed system is introduced graphically with a series of architecture diagrams with increasing granularity. Design considerations, trade-offs, and decisions are discussed subsequently. The report concludes with technology recommendations for implementing this system.

2 System Architecture Overview

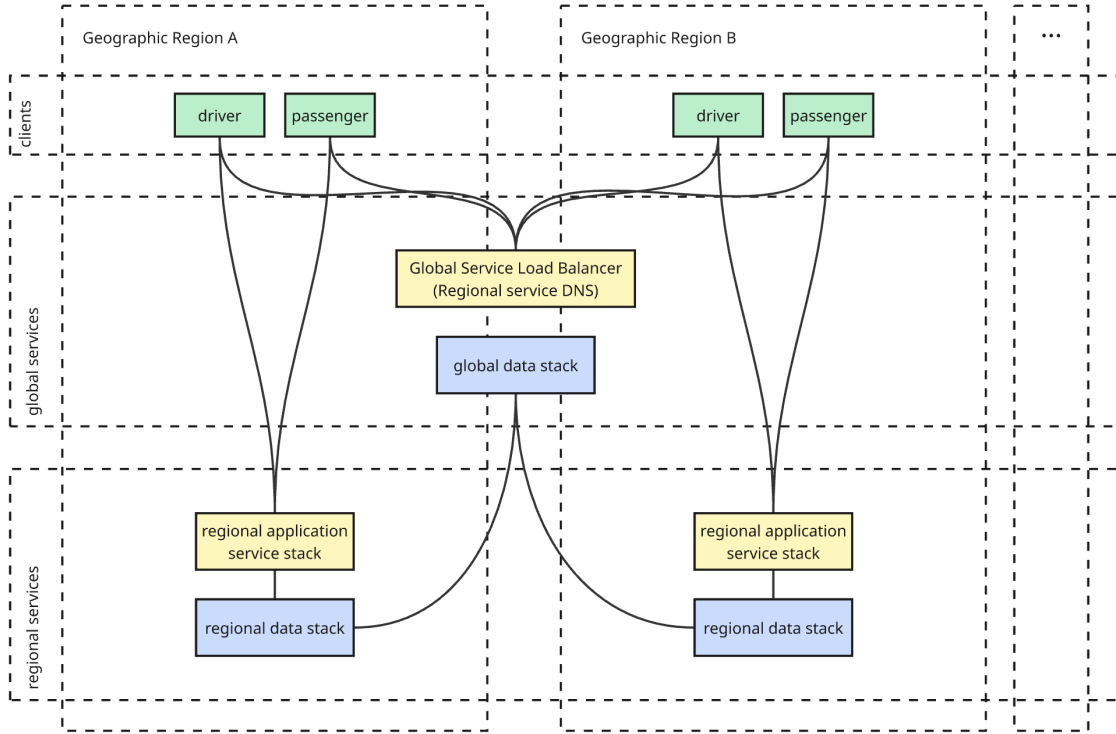


Figure 1: High-level system architecture.

At the highest level the system components can be categorized into three groups: clients, global services, and regional services. The personas of the clients may be either passengers or drivers and their interfaces may be native mobile applications or web browsers. The global services are responsible for authentication, authorization, account management, and regional service routing. The bulk of core functionality is provided by the regional services, of which there can be an arbitrary number of instances.

When clients begin interacting with the system they query the global service load balancer (GSLB) for determine the appropriate regional service endpoint with which to communicate. The GSLB's DNS resolution is governed by a geolocation routing policy that maps a client's current IP address to a regional service IP address. This policy may also consider availability, capacity, and load.

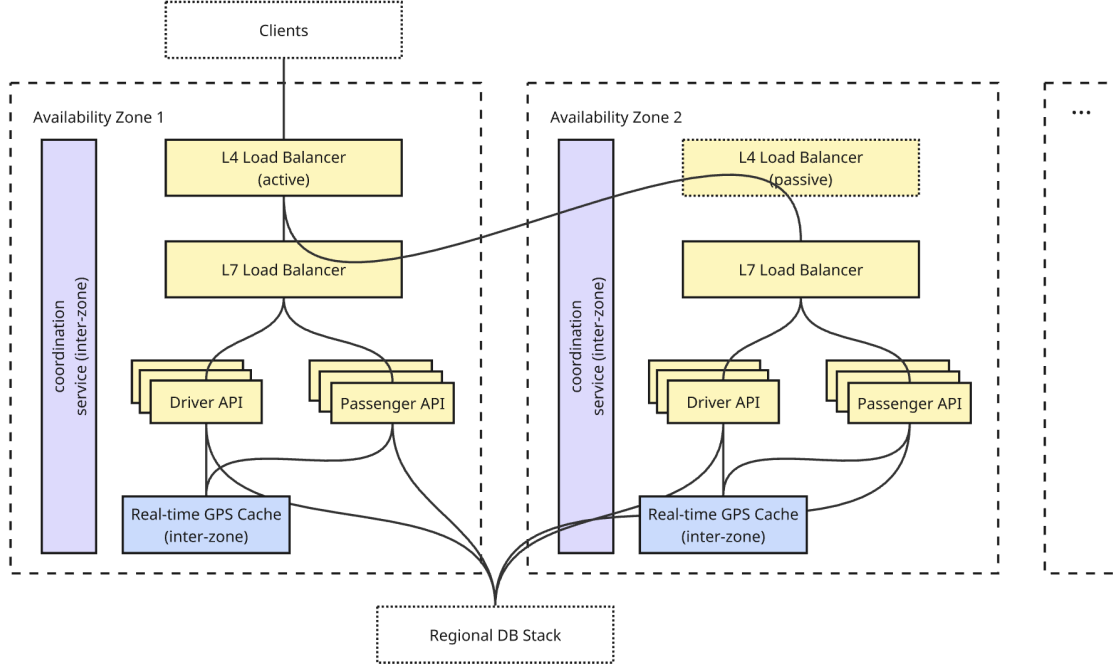


Figure 2: Regional application service stack architecture.

Once a client has been routed to a regional service endpoint a layer 4 load balancer (L4LB) proxies their requests to an availability zone (AZ). The purpose of the L4LB is to distribute load across AZs and adapt to outages in AZs. At any given time there is a single active L4LB in a region, backed by several passive instances that can take over should the active instance fail. Next, a layer 7 load balancer (L7LB) proxies requests to an appropriate application API based on the request path and headers. The L7LB is also responsible for TLS termination and rate limiting. This allows components deeper in the stack to focus on core functionality rather than security and load management.

Each application API has multiple instances running, with the specific number scaling with the current or anticipated load. The API services are stateless which allows them to be regarded as ephemeral. The diagram depicts two specific APIs for illustration purposes, but in reality there will likely be many more along with additional supporting microservices. Each region also has an inter-zone distributed cache for rapidly changing non-durable data such as GPS locations of users. Durable data is managed by regional data stacks. Finally, each region has a coordination service (CS) that spans AZs. The CS is responsible for leader election in the case of the L4LBs. It also provides service registration and discovery which enables the L7LB to route to a flexible pool of API instances.

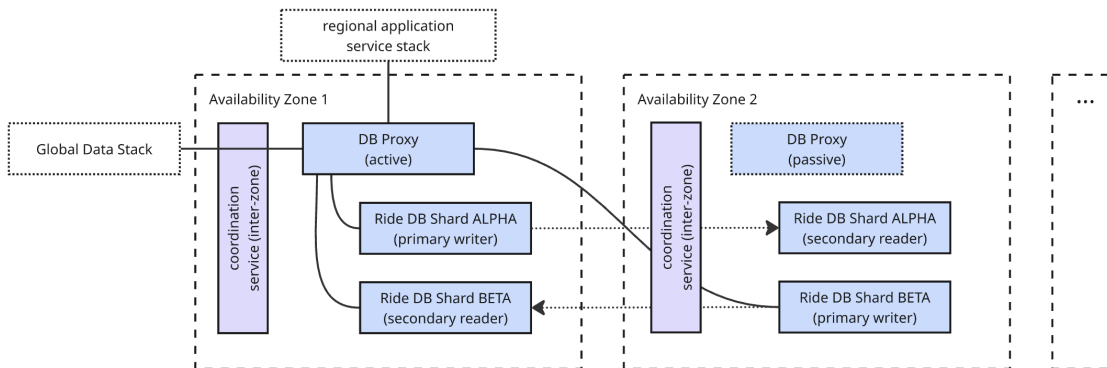


Figure 3: Regional data stack architecture.

Each regional data stack manages a cluster of high volume ride data, partitioned geospatially. We horizontally partition the data so that each regional data stack is independent from the others. Within each regional stack we shard records across primary writer instances in different AZs using a discrete global grid system (e.g. Uber’s H3) and consistent hashing. This approach gives us semantically meaningful shard keys that are also well distributed. Each shard (‘ALPHA’, ‘BETA’, etc.) is synchronously replicated to instances in other AZs for durability and fault tolerance. We depend on the DB technology to orchestrate this replication.

Applications are insulated from the complexity of the regional and global data stacks by a DB proxy (DBP). The DBP routes queries to the shard based on the hash key and current availability. It also lazily loads records from the global data stack when new clients use the service or existing clients move between regions.

Each regional data stack has a coordination service similar to those in the regional service stacks. Both the DBP and shard instances use the coordination service to elect leaders. The DBP also relies on the service for service discovery: it needs to immediately know when shard leadership changes.

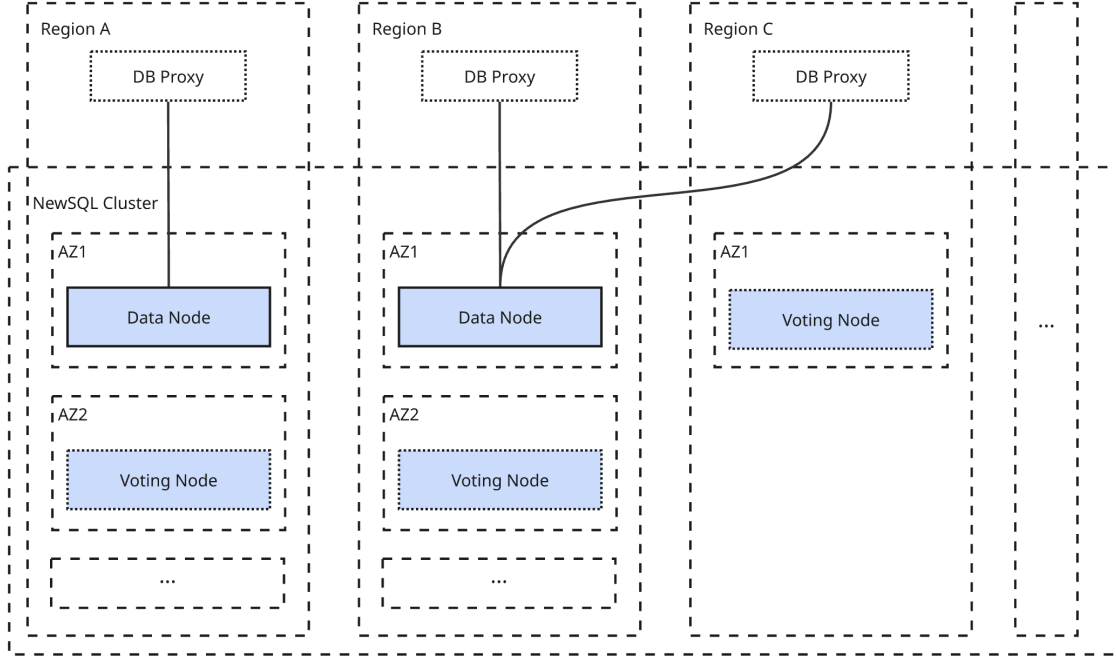


Figure 4: Global data stack architecture.

The global data stack is responsible for owning and managing durable data that is not specific to one single region. This data is much smaller in volume and slower changing than the regional ride-based data, but arguably more critical to the system’s operator: account information, payment information, and pricing rules for example. The global data stack is based around a single NewSQL cluster. This cluster has a number of replicated data nodes distributed across regions and AZs. This approach allows for quick reads and slow writes, which is fitting for the access pattern expected for this data.

3 Database Sharding

3.1 Sharding Strategies

3.1.1 Considered

- **Hash-based:** This simple sharding assigns each record to a shard by hashing a key (e.g. user ID) and taking the result modulo the shard count. Writes spread evenly and routing is cheap to compute. However, adding or removing a shard changes the modulus, forcing a reshuffle of existing data.
- **Range-based:** This sharding assigns contiguous key ranges (e.g. timestamps) to specific shards. Range scans are cheap and the shard boundaries are easy to reason about, but it is susceptible to hotspotting. For example a single popular range, like downtown at rush hour, can lead to a single shard absorbing a disproportionate amount of the load.

- **Consistent hashing:** This approach places shards and keys on the same hash ring, so adding or removing a shard only remaps the keys next to it on the ring rather than the whole dataset. It can still hotspot, since one shard may still own a disproportionate arc of the ring, but virtual nodes mitigate this by spreading each physical shard’s ownership across many smaller arcs.

3.1.2 Chosen

The data in this system is sharded at multiple levels in this system. First, data is partitioned by region. We assume that the number of regions have enough flexibility and scaling capacity that their count will rarely change. This is akin to range-based sharding as keys, or client IP addresses, should map to the same region. Second, within regional data stacks we shard data by geospatial index (H3) and consistent hashing. This allows the number of shards to scale in response to demand without reshuffling the entire dataset, while also giving us shard keys that are semantically meaningful and well distributed. As long as the granularity of the H3 cells is chosen appropriately, this approach will also avoid hotspotting.

4 Replication and Consistency Models

4.1 Consistency Model Comparison

4.1.1 Considered

- **Strong consistency:** All reads reflect the most recent write. Strong correctness guarantees and high latency
- **Eventual consistency:** Writes are acknowledged quickly and replicas converge over time. Reads may return stale data
- **Quorum-based consistency:** A write (or read) is acknowledged once a majority of replicas confirm it. A tunable hybrid of the preceding approaches.

4.1.2 Chosen

Each of these consistency models are represented in this system. We have strong consistency guarantees for ride matching data, as we want riders and drivers to have reliable experiences when being matched and paying for services. Low latency is important for that use case as well. We can accept the trade-off slightly slower writes because all replication happens in the the same region. The distributed GPS cache is eventually consistent, which is acceptable for data that is written frequently, read frequently, and does not involve accounting for real world resources: vehicles dispatched, rides cancelled, payment receipts, etc. Stale reads will not have a meaningfully negative impact on user experience. In return for the reduced consistency guarantee we get lower latency and very high throughput. This is ideal for the data that will be written orders of magnitude more often than other data types in the system. Finally, we distinguish between two categories of global data. Security-critical user account fields — credentials, account status, payment methods, and driver approval status — require

strong consistency globally. A deactivated driver who can still accept rides because a regional cache has not caught up is a safety issue, not a UX problem. These fields must always be read through to the global data stack, bypassing the regional DBP cache entirely. Profile data (display name, preferences, photo) can tolerate quorum-based consistency; stale reads have no meaningful consequence, and the latency cost of strong global consistency on every profile read is not justified.

5 Load Balancing

The system contains load balancers performing 3 distinct functions. The GSLB acts as a DNS resolver. Its only purpose is to direct clients to the correct region. It reforms this task by mapping client IP addresses to fixed virtual IP (VIP) addresses used by L4LBs in each region. When a L4LB wants to move from passive to active it assumes the VIP.

L4LBs are the gateways for all inbound client traffic to each region. They act as true load balancers, distributing load across available downstream resources as efficiently as possible. They do not process requests or even investigate payloads. They operate at the level of the TCP protocol so the CPU usage is kept to a minimum.

Like the L4LBs, the L7LBs proxy requests towards application services, but L7LBs perform more middleware actions on the requests. Working at the HTTP/application level, these load balancers have access to request paths, methods, and headers. They use this information to filter, modify, and route requests to different types of APIs.

6 Evaluation

CAP

Capacity

Latency

7 Technology Recommendations

Thus far the system has been described in terms hardware and software agnostic to the extent possible. The system as is can be implemented with a number of technologies. In the process of selecting specific environments and frameworks, some components may split into multiple, while others may be combined. What follows is an exercise that illustrates how this system might be implemented.

We have chosen to use open-source technologies where possible, to avoid vendor lock-in and to allow for greater flexibility in deployment. The following table summarizes the software recommendations for each system component, along with rationale for each.

7.1 Infrastructure

We will use a cloud provider for the underlying physical infrastructure. This allows us to get up and running quickly with the ability to scale along the way. The specific provider is

not important as no provider specific services will be used. (In reality, this may not be the best decision. This was done to allow for learning about open source technologies.)

The compute and networking resources needed to run the system will be provisioned with Terraform. After provisioning, the system will be deployed and configured with Ansible. These two tools allow us to define the system architecture with code instead of manually standing up the infrastructure: it’s fast, readable, and repeatable.

The app services (e.g. APIs) will require more active management than Ansible is intended to provide. There we will containerize the services and manage the containers with Kubernetes. Kubernetes can be configured to automatically scale with load and we also get metrics and logging out of the box.

7.2 Software

Table 1: Technology recommendations by system component.

Component	Recommendation	Rationale
GSLB	PowerDNS [4]	Authoritative DNS server with a Lua-based routing API and REST management interface. Supports geolocation-based resolution and health-check-driven failover.
L4LB	HAProxy [3]	High-throughput TCP proxy with active/passive failover support. Lightweight enough to not add meaningful latency on the critical path.
L7LB	Envoy [2]	HTTP routing by path and header, TLS termination, rate limiting, and built-in observability.
CS	etcd	Strong consistency via Raft consensus, purpose-built for leader election and service discovery, and well-proven at scale (it powers Kubernetes).
DBP	Custom service	The required routing logic (consistent hashing over H3 keys, shard discovery via the coordination service, lazy cross-region loading) is application-specific and not handled by off-the-shelf proxies.
GPS Data	Redis	Sub-millisecond reads and writes, native geospatial commands (<code>GEOADD</code> , <code>GEORADIUS</code>), and per-key TTL for automatic expiry of stale locations.
Pricing Rules	Redis	Small, read-heavy dataset that fits entirely in memory. Replication ensures regional availability and TTL-based invalidation handles infrequent updates.

Component	Recommendation	Rationale
Active Trip Data	Apache Cassandra	High write throughput, tunable consistency (supports the quorum model chosen for trip records), and horizontal scaling across AZs without a single point of coordination.
User Accounts	CockroachDB [1]	Distributed NewSQL database with serializable isolation across nodes, satisfying the strong consistency requirement for security-critical fields. Geo-partitioning anchors each user's record to their home region for fast local reads, while still coordinating writes globally when security-critical fields change.
Trip History	PostgreSQL	Declarative time-based range partitioning maps directly to the range-sharding strategy chosen for this data type, and mature SQL support covers the analytical and auditing queries this data is accessed for.

References

- [1] CockroachDB – the distributed SQL database. <https://www.cockroachlabs.com/>. Accessed: 2026-06-30.
- [2] Envoy – cloud-native high-performance edge and service proxy. <https://www.envoyproxy.io/>. Accessed: 2026-06-30.
- [3] HAProxy – the reliable, high performance TCP/HTTP load balancer. <https://www.haproxy.org/>. Accessed: 2026-06-30.
- [4] PowerDNS – the authoritative DNS server. <https://www.powerdns.com/>. Accessed: 2026-06-30.